

GestorOps

Documento Estratégico v1.0

Arquitectura · Producto · Roadmap · Seguridad · UX

Fecha: Mayo 2026 · Versión: 1.0 · Confidencial

Este documento define la estrategia completa, arquitectura técnica y plan de desarrollo de un sistema operativo interno para agencias tecnológicas y freelance. Cubre desde el análisis de producto hasta el roadmap de implementación.

Índice

Sección	Contenido
Propuestas de nombre	15 opciones de branding modernas y tecnológicas
Fase 1 — Análisis de Producto	Problema, valor, usuarios, MVP, riesgos
Fase 2 — Arquitectura	Stack técnico, diagrama, análisis crítico
Fase 3 — Estructura Funcional	Módulos, objetivos, prioridades
Fase 4 — Base de Datos	Entidades, relaciones, datos sensibles
Fase 5 — Seguridad	Vault, cifrado, roles, errores a evitar
Fase 6 — UX / Visual	Paleta, layout, patrones, micro-detalles
Fase 7 — Roadmap	Bloques, fases, dependencias, orden
Fase 8 — Visión Futura	Evolución SaaS, límites, oportunidades
Resumen Ejecutivo	Decisiones clave consolidadas

Propuestas de Nombre

15 opciones modernas, tecnológicas y escalables para SaaS B2B.

#	Nombre	Concepto
1	Opsra	Operations + aura. Breve, tech, memorable.
2	Velox	Velocidad operativa. Limpio, serio, pronunciable.
3	Nexlink	Nexo entre cliente y operación técnica.
4	Clarix	Claridad operativa. Elegante, tech, distintivo.
5	Orbix	Centro de operaciones orbital. Moderno.
6	Stackly	Stack de operaciones. Técnico, familiar.
7	Corevex	Core operations. Serio, escalable.
8	Opflow	Operations flow. Directo, limpio, intuitivo.
9	Lumio	Luminoso, claro, operativo. Fácil de recordar.
10	Netron	Red operativa técnica. Serio y tech.
11	Zephyr	Ligero, rápido, fluido. Premium natural.
12	Klarix	Claridad + operación. Variante más agresiva.
13	Pulseo	Pulso de la operación. Vivo, dinámico.
14	Aevio	Eterno + operativo. Elegante, único.
15	Vexor	Vector operativo. Técnico, escalable, SaaS-ready.

Recomendación principal: CLARIX o ORBIX — cortos, técnicos, con buena disponibilidad potencial de dominio, sin ser genéricos.

Análisis de Producto

El problema real

Una agencia tecnológica pequeña o freelance senior acumula clientes de forma orgánica. Con 10 clientes ya existe caos: contraseñas dispersas, mantenimientos sin registro, incidencias por WhatsApp, renovaciones de dominio olvidadas, proyectos con estados informales y documentación técnica dispersa. Con 20 clientes el caos es operativo. Con 40, es un riesgo real de negocio.

Problema central

Ausencia de un sistema de registro operativo profesional y centralizado, pensado específicamente para agencias tech y freelance.

Valor que aporta

- Control total: todo sobre cada cliente en un único lugar.
- Reducción de errores: renovaciones, mantenimientos y accesos trazados.
- Velocidad operativa: encontrar cualquier dato en 3 segundos.
- Historial técnico: saber qué se hizo, cuándo y por qué.
- Profesionalidad interna: operar como una empresa seria.
- Escalabilidad del equipo: onboarding de un nuevo técnico en minutos.

Usuario objetivo

Perfil	Descripción
Freelance senior	10–50 clientes, opera solo o con 1-2 colaboradores
Microagencia tech	3–15 personas, clientes SMB, servicios recurrentes
Consultora técnica	Gestión de proyectos + mantenimientos complejos

Funcionalidades críticas — MVP

- Gestión de clientes con ficha completa
- Gestión de proyectos por cliente
- Registro de accesos técnicos cifrados
- Control de mantenimientos mensuales
- Registro de incidencias
- Renovaciones con alertas
- Dashboard operativo resumen

Funcionalidades que pueden esperar (v2+)

- Analítica avanzada
- Automatizaciones y notificaciones push
- Integración con herramientas externas (Slack, Linear, etc.)
- Portal de cliente externo
- Facturación o integración contable
- Gestión multiusuario con permisos granulares
- App móvil

Riesgos de diseño inicial

Riesgo	Descripción	Mitigación
Sobre-ingeniería prematura	Construir 20 módulos antes de validar los 5 críticos	MVP disciplinado
Diseño de BD rígido	Tablas sin flexibilidad que bloquean el crecimiento	JSONB para metadatos extensibles
Seguridad superficial	Guardar credenciales sin cifrado real	AES-256 obligatorio desde día 1
UX compleja	Dashboard con demasiada información	Jerarquía visual estricta
Sin separación de contextos	Datos de clientes mezclados	Isolation por client_id en todas las tablas
Dependencia única de Supabase	Sin estrategia de backup externa	Backups automáticos + exportación manual

Arquitectura del Sistema

Stack recomendado

Capa	Tecnología	Rol
Frontend	Next.js 14+ (App Router) + React + TypeScript	SSR, RSC, routing
Estilos	Tailwind CSS + shadcn/ui	Diseño, componentes
Backend	Next.js API Routes / Server Actions + Supabase Edge Functions	Lógica de negocio
Auth	Supabase Auth (JWT + RLS)	Autenticación y autorización
Base datos	PostgreSQL vía Supabase	Persistencia principal
Secretos	Vault (Supabase) + AES-256 app level	Credenciales cifradas
Storage	Supabase Storage	Archivos y documentos
Despliegue	Vercel	Edge deploy, CI/CD
Monitoreo	Vercel Analytics + Sentry	Observabilidad

Análisis crítico del stack

Next.js + App Router

Ventajas: SSR, RSC, API nativa, ecosistema enorme, Vercel-native. Server Actions simplifican el backend para operaciones CRUD.

Limitaciones: App Router tiene rough edges. Curva de aprendizaje de RSC es real.

Supabase

Ventajas: PostgreSQL real, Auth integrado con RLS, Storage, Realtime, Vault para secretos, excelente DX, muy rápido para MVP.

Limitaciones: Free tier pausa proyectos inactivos. Producción requiere plan Pro (\$25/mes). Vendor lock-in moderado — mitigado usando PostgreSQL estándar.

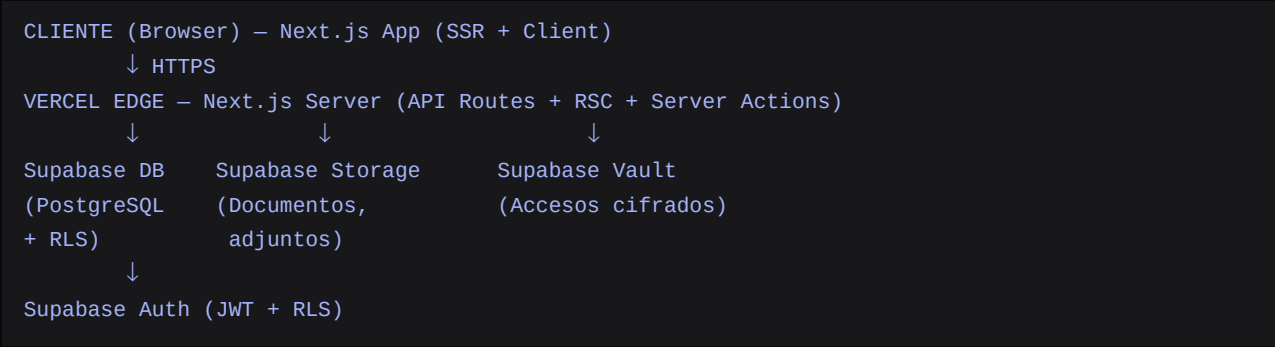
Tailwind + shadcn/ui

Ventajas: shadcn/ui son componentes que se copian al proyecto — control total, accesibilidad nativa (Radix UI), altamente customizable.

Vercel

Ventajas: Zero-config deployment, Edge Network, preview deployments, integración nativa con Next.js. Viable en free tier para uso interno inicial.

Diagrama de Arquitectura Conceptual



Seguridad de capa de arquitectura

Capa	Tecnología	Función
1	HTTPS (Vercel)	Transporte cifrado — automático
2	Supabase Auth	JWT, sesiones, refresh tokens
3	RLS PostgreSQL	Autorización a nivel fila en BD
4	Cifrado en reposo	Supabase cifra la BD base
5	AES-256-GCM app	Vault de accesos técnicos
6	Audit log inmutable	Trazabilidad de acciones
7	Rate limiting	Protección brute force

Estructura Funcional — Módulos

Dashboard

CRÍTICA Bloque 1

Vista operativa en tiempo real del estado del negocio.

- Incidencias abiertas (crítico primero)
- Mantenimientos pendientes del mes
- Renovaciones próximas (30/15/7 días)
- Proyectos activos por estado
- Últimas actividades del sistema

Relaciones: Agrega datos de todos los módulos del Bloque 1

Clientes

CRÍTICA Bloque 1

CRM interno completo para registrar todo sobre cada cliente.

- Nombre, empresa, sector, tipo (activo/inactivo/prospect)
- Contactos múltiples (técnico, comercial, decisor)
- Notas internas y tags
- Fecha de alta y último contacto

Relaciones: Todos los módulos tienen client_id como FK

Proyectos

CRÍTICA Bloque 1

Gestión del ciclo de vida de proyectos técnicos.

- Estados: proposal → active → paused → completed → cancelled
- Stack técnico, alcance, presupuesto
- Fecha inicio / fin estimado / fin real
- Hitos y tareas asociadas

Relaciones: Pertenece a Cliente. Relacionado con Incidencias y Documentación

Mantenimientos

CRÍTICA Bloque 1

Control de contratos de mantenimiento recurrente por cliente.

- Contratos con periodicidad configurable
- Instancias mensuales: pending → in_progress → completed → invoiced
- Historial de acciones realizadas por mes
- Notas técnicas del periodo

Relaciones: Pertenece a Cliente

Incidencias

ALTA Bloque 1

Registro y resolución de problemas técnicos.

- Estados: open → in_progress → waiting_client → resolved → closed
- Prioridad: critical / high / medium / low
- Tiempo invertido y resolución final
- Vinculable a proyecto o cliente directamente

Relaciones: Pertenece a Cliente, opcional a Proyecto

Accesos Técnicos

CRÍTICA Bloque 1

Vault cifrado de credenciales y accesos por cliente. Módulo más sensible del sistema.

- Tipos: hosting, BD, plataformas, APIs, dominios, email, custom
- Campos cifrados con AES-256-GCM a nivel aplicación
- Log de accesos: quién vio qué y cuándo
- Oculto por defecto / mostrar bajo demanda

Relaciones: Pertenece a Cliente. Tiene access_log propio

Renovaciones

ALTA Bloque 1

Control de vencimientos de dominios, hosting, licencias y certificados.

- Tipos: dominios, hosting, licencias, SSL, contratos
- Alertas: 90/30/15/7/1 días antes
- Estado: active / expired / cancelled
- Proveedor, coste, notas

Relaciones: Pertenece a Cliente

Documentación

MEDIA Bloque 2

Base de conocimiento técnica interna.

- Tipos: documentación de proyecto, manuales, procedimientos, notas
- Estructura jerárquica: colección > documento > secciones
- Editor de texto enriquecido (Markdown)

Relaciones: Vinculable a Cliente o Proyecto

Infraestructura

MEDIA Bloque 2

Registro del stack técnico de cada cliente (qué servicios tiene activos).

- Servidores, bases de datos, CDNs, plataformas SaaS
- Estado, proveedor, descripción
- Diferente de Accesos: descriptivo, no operativo

Relaciones: Pertenece a Cliente

Historial Técnico

MEDIA Bloque 2

Log cronológico de todo lo realizado para cada cliente.

- Generación automática por eventos del sistema
- Entradas manuales por el técnico
- Formato: [fecha] [tipo] [descripción] [módulo] [autor]

Relaciones: Fuente: activity_log. Vinculado a Cliente

Tareas

MEDIA Bloque 2

Gestión ligera de work items internos del equipo técnico.

- Título, descripción, estado, prioridad
- Asignación, fecha límite
- Vinculable a Cliente o Proyecto

Relaciones: Opcional: Cliente, Proyecto

Analítica

BAJA Bloque 3

Visibilidad sobre el negocio y la operación.

- Incidencias por cliente/mes
- Tiempo de resolución promedio
- Estado de mantenimientos del mes
- Proyectos activos vs completados

Relaciones: Agrega datos de todos los módulos

Usuarios y Permisos

ALTA Bloque 0

Gestión de acceso al sistema con roles y permisos.

- Roles: admin | technician | readonly
- Supabase Auth + RLS como base
- Preparado para permisos granulares en v2

Relaciones: Transversal — afecta todos los módulos

Diseño de Base de Datos

Jerarquía de entidades

```
organizations
  ■■■ users          (técnicos y admins)
  ■■■ clients        (clientes gestionados)
    ■■■ projects
    ■■■ maintenances
    ■ ■■■ maintenance_entries
    ■■■ incidents
    ■■■ technical_accesses
    ■ ■■■ access_log
    ■■■ renewals
    ■■■ infrastructure_items
    ■■■ documents
    ■■■ activity_log  (append-only)
  ■■■ tasks
  ■■■ notifications
```

Tablas principales

Tabla	Campos principales
clients	id, org_id, name, company, sector, type, email, phone, website, notes, status, created_at, updated_at, created_by
projects	id, client_id, org_id, title, description, tech_stack (jsonb), status, priority, start_date, estimated_end_date, actual_end_date, budget, notes, tags[]
maintenances	id, client_id, org_id, name, description, type, price, billing_period, status, start_date
maintenance_entries	id, maintenance_id, period (YYYY-MM), status, work_done, hours_spent, completed_at, completed_by
incidents	id, client_id, project_id?, org_id, title, description, priority, status, resolution_notes, hours_spent, opened_at, closed_at, assigned_to
technical_accesses	id, client_id, org_id, type, name, url, username_encrypted, password_encrypted, notes_encrypted, last_verified_at, status
renewals	id, client_id, org_id, name, type, provider, renewal_date, cost, status, notify_days[]
infrastructure_items	id, client_id, org_id, category, name, provider, url, description, status, metadata (jsonb)
documents	id, client_id?, project_id?, org_id, title, content, type, tags, created_by
activity_log	id, org_id, entity_type, entity_id, action, description, metadata (jsonb), performed_by, created_at [APPEND-ONLY]
tasks	id, org_id, client_id?, project_id?, title, description, status, priority, due_date, assigned_to

Tabla	Campos principales
users	id (=Supabase Auth uid), org_id, full_name, email, role, avatar_url, status, last_login

Datos sensibles y protección

Dato	Sensibilidad	Protección
Contraseñas de cliente	CRÍTICA	AES-256-GCM, cifrado aplicación
Tokens / API keys	CRÍTICA	Mismo cifrado, log de acceso obligatorio
Datos de cliente (PII)	ALTA	RLS en Supabase por org
Notas técnicas	MEDIA	RLS por org / rol
Historial de actividad	ALTA (integridad)	Append-only — sin DELETE permitido
Documentación	MEDIA	RLS por org y rol

Preparación para crecer

- org_id presente en todas las tablas críticas desde el día 1 — multitenancy listo en el modelo.
- metadata (jsonb) en tablas clave para campos extensibles sin migración.
- activity_log como tabla de eventos central — base para auditoría y analítica futura.
- Índices en columnas frecuentes: client_id, status, renewal_date, created_at.
- tags (text[]) en proyectos y documentos para filtrado rápido.

Seguridad y Accesos

CRÍTICO: Estás gestionando credenciales de terceros. Una brecha aquí no afecta solo a tu empresa — afecta a todos tus clientes. Esto requiere una postura de seguridad seria desde el día 1.

Arquitectura de seguridad en capas

Capa	Tecnología	Función
1 — Transporte	HTTPS (Vercel automático)	Cifrado en tránsito
2 — Autenticación	Supabase Auth + JWT	Identidad de usuario
3 — Autorización	RLS en PostgreSQL	Control de acceso a nivel fila
4 — Cifrado reposo	Supabase + AES-256 app	Protección de datos almacenados
5 — Vault de accesos	AES-256-GCM derivado servidor	Credenciales de clientes
6 — Audit log	activity_log inmutable	Trazabilidad completa
7 — Rate limiting	Supabase Auth + Vercel WAF	Protección brute force

Roles del sistema

Rol	Capacidades
admin	Acceso total. Puede crear usuarios, ver todos los módulos, acceder al vault completo.
technician	Acceso operativo. Puede ver/editar clientes, proyectos, incidencias. Acceso limitado al vault.
readonly	Solo lectura. Para colaboradores externos o revisores.

Gestión del Vault de Accesos Técnicos

Cómo NO hacerlo

- Guardar contraseñas en texto plano en la BD
- Cifrar solo en BD sin cifrado a nivel aplicación
- Exponer credenciales descifradas en respuestas de API
- Loggear credenciales en errores o consola
- Enviar credenciales por correo o Slack

Flujo correcto de cifrado

```
CLAVE = derivada de ENCRYPTION_SECRET (Vercel env var – nunca en código)
DATO_CIFRADO = AES-256-GCM(dato_plano, CLAVE, IV_aleatorio)
BD guarda: dato_cifrado + IV (nunca la clave)
```

Flujo de acceso:

1. Usuario autenticado solicita ver credencial
2. Server Action valida JWT + RLS
3. Servidor descifra en memoria (nunca persistido descifrado)
4. Responde solo el dato – registra en access_log: quién, qué, cuándo
5. Campos en UI: ocultos por defecto, copiar sin mostrar visualmente

Errores críticos a evitar

Error	Consecuencia	Solución
RLS desactivado en alguna tabla	Filtración cross-tenant	RLS en TODAS las tablas con org_id
Secret en código fuente	Compromiso completo del sistema	Solo env vars en Vercel — nunca en repo
Sin log de acceso a credenciales	Sin trazabilidad forense	access_log obligatorio desde v1
Backup sin cifrar	Exposición de datos sensibles	Supabase cifra backups automáticamente
Sin expiración de sesión	Sesiones infinitas reutilizables	JWT con TTL razonable (7 días)
Errores verbosos en producción	Leak de información técnica	Sentry solo en server, mensajes genéricos al cliente

Experiencia Visual y UX

Filosofía de interfaz

Referentes: Linear para densidad informativa y velocidad. Vercel para limpieza visual. Notion para flexibilidad. Stripe Dashboard para jerarquía de datos operativos.

Sistema de color

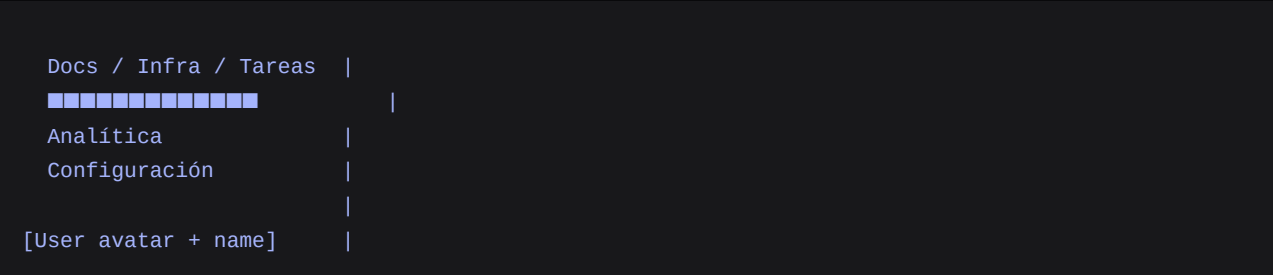
Modo oscuro por defecto. El 90% de usuarios técnicos trabajan en dark mode. No ofrecer modo claro en v1 — añade complejidad sin valor inicial.

Estructura de Layout

```

SIDEBAR (240px fijo) | MAIN CONTENT
                        |
[Logo / Org name]    | [Breadcrumb + Page title]
                        | [Action bar]
Navigation:           |
  Dashboard            | [Content area]
  Clientes             |
  Proyectos            |
  Mantenimientos       |
  Incidencias          |
  Accesos              |
  Renovaciones         |
  ■■■■■■■■■■■■■■■■   |

```



Patrones de UX clave

Patrón	Descripción
Command Palette (Cmd/Ctrl+K)	Búsqueda global: clientes, proyectos, accesos, documentos. Navegación sin sidebar.
Listas con alta densidad	Tablas con filas compactas (44px). Acciones en hover. Ordenación por columna.
Formularios en modal/slideoveer	No páginas nuevas — el contexto no se rompe. Validación inline.
Skeleton screens	Para listas y cards durante carga. Sin spinners centrales.
Toast notifications	Top-right, 3s para éxito, persistente para errores. Deshacer donde sea viable.
Badges de alerta en sidebar	Renovaciones vencidas, incidencias críticas, mantenimientos pendientes.
Confirmaciones destructivas	Input de texto para operaciones críticas — no solo botón de confirmación.
Empty states con CTA	No pantallas vacías — siempre orientar la siguiente acción.

Roadmap de Desarrollo

Principio: Cada bloque produce un sistema usable y valioso al terminar, no depende del siguiente bloque para aportar valor.

Bloque 0 — Fundación

Duración estimada: 3–5 días

- Setup del repositorio, Next.js + Tailwind + TypeScript + shadcn/ui
- Configuración Supabase (proyecto, auth, DB)
- Sistema de autenticación completo (login, sesión, logout, rutas protegidas)
- Layout base (sidebar, navegación, estructura de páginas)
- Design tokens y componentes base (Button, Input, Card, Badge, Modal, Table)
- CI/CD básico con Vercel

Resultado: Shell profesional de la aplicación. Login funciona. Navegación funciona. Sin datos reales.

Bloque 1 — Núcleo Operativo

Duración estimada: 2–3 semanas

- 1. Clientes — CRUD completo con ficha detallada
- 2. Proyectos — CRUD + estados + relación con cliente
- 3. Mantenimientos — Contratos + entradas mensuales
- 4. Incidencias — CRUD + estados + prioridad
- 5. Renovaciones — CRUD + fechas + alertas visuales
- 6. Accesos Técnicos — Vault cifrado + audit log
- 7. Dashboard — Vista agregada de los módulos anteriores

Resultado: Sistema completamente usable en el día a día. Reemplaza Notion/Excel/1Password.

Bloque 2 — Profundidad Operativa

Duración estimada: 2 semanas

- Documentación — Editor markdown + organización jerárquica
- Infraestructura — Registro de servicios por cliente
- Historial técnico — Log automático + manual de eventos
- Tareas — Gestión ligera de work items
- Command Palette — Búsqueda global Cmd+K
- Notificaciones internas — Alertas de renovaciones y pendientes

Resultado: Sistema completo y profundo. Toda la información operativa centralizada.

Bloque 3 — Calidad y Polish

Duración estimada: 1 semana

- Analítica básica (métricas clave del negocio)
- Exportación de datos (CSV para backups manuales)
- Gestión de usuarios (si hay equipo)
- Onboarding / empty states pulidos
- Performance audit y optimización
- Accesibilidad básica

Resultado: Producto terminado y pulido, listo para uso profesional diario estable.

Dependencias técnicas críticas

Auth		Todo lo demás
Clientes		Proyectos, Mantenimientos, Incidencias
Proyectos		Incidencias, Documentación
Cifrado (Accesos)		Vault de accesos técnicos
Activity Log		Historial, Analítica futura

Visión Futura

Trayectoria natural del producto

Fase actual — Herramienta interna

Aplicación que el equipo pequeño usa internamente para operar mejor. No necesita ser perfecta — necesita ser útil.

Evolución 1 — Agencia con equipo (5–10 personas)

Roles y permisos granulares, asignaciones por técnico, métricas de productividad, comunicación interna básica. No requiere rediseño — modelado desde el inicio con `org_id` y `assigned_to`.

Evolución 2 — Plataforma multiagencia (SaaS)

El mercado hispanohablante tiene decenas de miles de agencias con este problema. Requiere: onboarding automático, planes + facturación (Stripe), aislamiento entre orgs (ya preparado). Precio potencial: \$29–\$49/mes early adopter, \$59–99/mes en versión madura.

Evolución 3 — Centro operativo inteligente

Integración con Linear/GitHub, webhooks de Vercel para actualizar estados automáticamente, alertas por email/Slack/WhatsApp, OCR de facturas para registrar renovaciones, API pública.

Evolución 4 — Portal de cliente

Vista externa de solo lectura para que el cliente vea el estado de sus proyectos e incidencias. Comunicación cliente-agencia directamente en el sistema. Requiere subdominio separado y rol `client_user`.

Límites reales del stack

Escenario	Límite	Solución
>100 usuarios simultáneos	Supabase Pro escala bien	Pro plan o Supabase Dedicated
>500k rows en tablas activas	Rendimiento requiere índices cuidados	Normalizar + índices + particionado
Requisito GDPR estricto	Supabase EU region disponible	Seleccionar región EU al crear proyecto
Tiempo real crítico	Supabase Realtime funciona hasta cierto punto	Arquitectura de eventos dedicada
Compliance financiero	Fuera del scope	No añadir facturación sin auditoría de seguridad

Oportunidad de negocio

Dimensión	Análisis
Mercado objetivo	Agencias tech + freelance senior hispanohablantes — decenas de miles
Problema	Concreto, doloroso y costoso en tiempo y errores humanos
Competencia	Herramientas genéricas (Notion, Asana, HubSpot) no resuelven el caso específico
Poder adquisitivo	\$49–\$99/mes razonable para una agencia con 20+ clientes
Barrera de entrada	Baja para el producto, alta para el nicho — requiere conocimiento del sector

Resumen Ejecutivo

Dimensión	Decisión
Stack	Next.js + Supabase + Vercel + Tailwind + shadcn/ui
Arquitectura	Monolito modular con API Routes / Server Actions
Auth	Supabase Auth + RLS (Row Level Security)
Seguridad crítica	AES-256-GCM a nivel aplicación para el vault de accesos
Diseño	Dark mode, minimalista denso, inspirado en Linear
MVP	Bloque 0 + Bloque 1 = sistema usable y completo
Tiempo a MVP usable	3–4 semanas de desarrollo continuo
Escalabilidad	Multitenancy preparado desde el diseño inicial (org_id)
Nombre recomendado	CLARIX o ORBIX

Próximos pasos antes de escribir código

Validar las siguientes decisiones antes de iniciar el desarrollo:

- Modelo de datos: confirmar que las entidades y relaciones cubren la operativa real.
- Módulos del Bloque 1: verificar que son los 7 correctos para el MVP.
- Diseño visual: confirmar dirección dark mode / Linear-inspired.
- Nombre: seleccionar uno y verificar disponibilidad de dominio.
- Stack: confirmar que Next.js + Supabase + Vercel es el camino correcto.

Con estas decisiones tomadas, el Bloque 0 puede comenzar con claridad total y el sistema estará operativo en menos de un mes.